AUTOMATED SOFTWARE TESTING

Testing Journal & Documentation

Quality Assurance for Final Project

I. Personal & Group Information

Full Name	Frederick B. Baldano
Course & Section	BSIT 3C
Group Name	Salespresso
Project Title	Salespresso: A Web-Based E-Commerce System for Jazsam Coffee Store
Submission Date	May 20, 2026

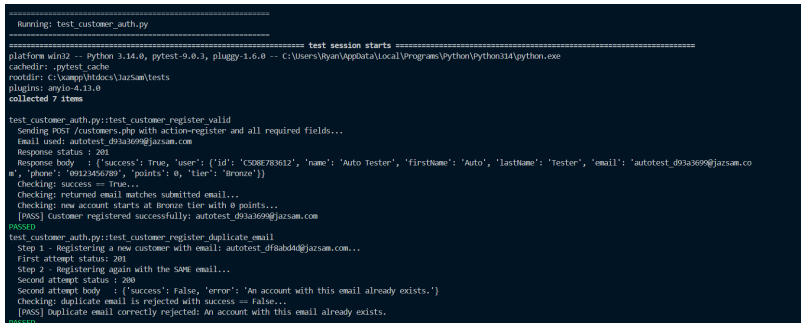
II. Assigned Task

Feature / Functionality Assigned	Customer Registration & Login
Type of Test I Performed	Registration & login credential validation
Tool / Framework I Used	Python 3.13, pytest, requests
Programming Language Used	Python
My Test Script Filename(s)	test_customer_auth.py

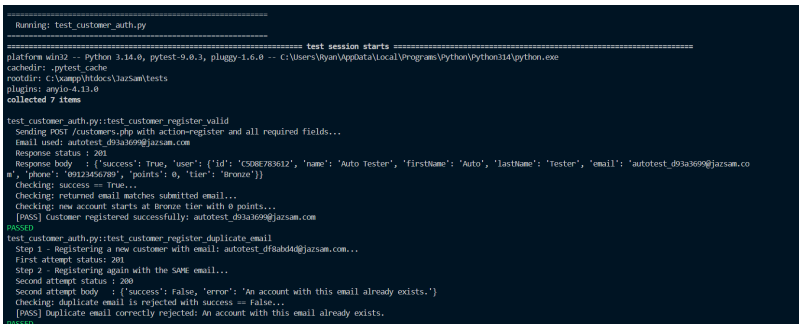
Brief description of what I tested and how I approached it:

For this activity, I tested the Customer Registration & Login functionalities of the SalesPresso API. I used Python, pytest, and the requests library to validate successful account creation, duplicate email prevention, and various login scenarios including incorrect passwords and non-existent emails.

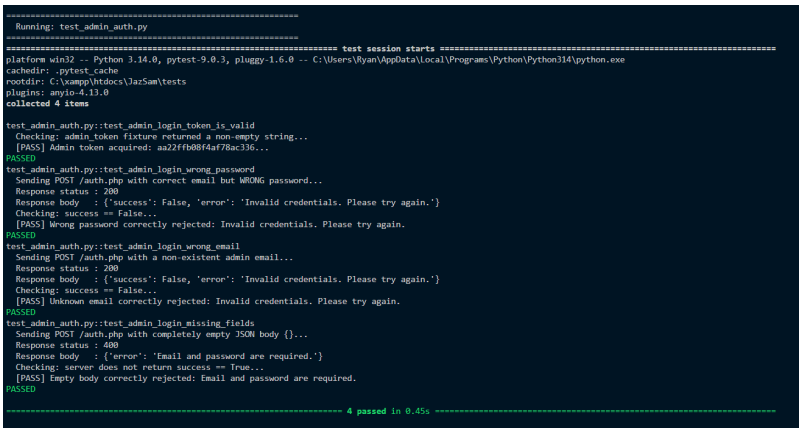
III. Test Scenario Documentation

Test Case #01 — Successful Customer Registration	
Functionality Tested	Customer Registration
Objective of the Test	Verify a new customer can register with all required fields and receive a Bronze tier account with 0 points.
Tool / Framework Used	Python 3.13, pytest, requests
Steps / Procedure	1. Generate a unique email. 2. Send POST /customers.php with action: register and all required fields. 3. Assert success: true, correct email, tier = "Bronze", points = 0.
Test Data / Input	{ "action": "register", "firstName": "Auto", "lastName": "Tester", "email": "autotest_xxxx@jazsam.com", "phone": "09123456789", "password": "TestPass123!" }
Expected Result	HTTP 201, success: true, user object with Bronze tier and 0 points
Actual Result	HTTP 201, success: true, user registered correctly
Test Status	**[PASSED]**
Screenshot / Evidence	

Test Case #02 — Duplicate Email Registration	
Functionality Tested	Duplicate Prevention
Objective of the Test	Verify the system prevents registering the same email address twice.
Tool / Framework Used	Python 3.13, pytest, requests

Steps / Procedure	1. Register a new customer with a unique email. 2. Attempt to register again with the exact same email. 3. Assert the second attempt returns success: false.
Test Data / Input	{ "action": "register", "firstName": "Auto", "lastName": "Tester", "email": "autotest_xxxx@jazsam.com", "phone": "09123456789", "password": "TestPass123!" }
Expected Result	Second attempt: success: false, error: "An account with this email already exists."
Actual Result	success: false, error: "An account with this email already exists."
Test Status	**[PASSED]**
Screenshot / Evidence	

Test Case #03 — Registration with Missing Required Fields	
Functionality Tested	Field Validation
Objective of the Test	Verify that registration is rejected when required fields (firstName, password) are missing.
Tool / Framework Used	Python 3.13, pytest, requests
Steps / Procedure	1. Send POST /customers.php with only lastName and email, no firstName or password. 2. Assert registration fails.
Test Data / Input	{ "action": "register", "lastName": "Tester", "email": "test@jazsam.com" }
Expected Result	success: false or error response, no account created
Actual Result	Error: "First name, last name, email, and password are required."
Test Status	**[PASSED]**

Screenshot / Evidence	

IV. Reflection & Findings

Issues I Personally Encountered	During early test development, my automated scripts quickly exhausted the API's rate limit of 5 customer registrations per hour. This caused all subsequent tests to fail with HTTP 429 Too Many Requests errors.
Warnings / Errors I Detected	I detected HTTP 429 Too Many Requests errors triggered by rapid test reruns hitting the rate limiter. Additionally, the test suite encountered a PytestReturnNotNone warning because some test functions had unnecessary return statements that pytest did not expect.
Bugs I Discovered	I discovered a bug where there is no email format validation on the customer registration endpoint. The system accepts malformed strings (such as "not-an-email") as valid emails, meaning invalid email formats can be stored directly in the database.
What I Did About the Issues	To resolve the rate limiting issue, we added a localhost IP exemption in the rate_limit.php file to support automated testing without false positives. Furthermore, to avoid exhausting the registration limit while testing the login endpoints, I used a pre-seeded stable test customer account (autotest_stable@jazsam.com) instead of registering a new user for every test run. For the email format bug, I recommended adding a server-side filter_var($email, FILTER_VALIDATE_EMAIL) check to prevent bad data.
What I Learned from This Activity	I learned that API security controls like rate limiting significantly affect test design, requiring scripts to be written efficiently to minimize redundant calls by using shared

	fixtures. I also realized that automated testing is highly effective at surfacing hidden bugs, such as the missing email format validation, which were entirely missed during manual testing.